

PLP Admissions — Database Q&A

Comprehensive reference for the interview / panel Q&A round

Verified against `baseline` after the latest project zip merge

A defensive walkthrough of the database — schema, integrity, transactions, performance, backup, security. Every claim ties back to actual code or schema definitions. If anyone challenges a claim during Q&A, **the code is the source of truth.**

A. Schema & Design

A1. Why MySQL / MariaDB?

The system targets MariaDB / MySQL because:

- It's the de-facto standard for vanilla-PHP deployments and the default in most LAMP-style hosting plans available to PLP.
- It supports the full set of features we rely on: transactions (`BEGIN` / `COMMIT` / `ROLLBACK`), row-level locking with `SELECT ... FOR UPDATE`, foreign-key constraints, unique indexes, `utf8mb4` for full Unicode (including emoji-safe display names).
- The InnoDB engine gives us crash recovery and ACID semantics out of the box.

A2. How many tables are there?

26 InnoDB tables declared in `database/schema.sql`, plus a small set of auxiliary tables (`exam_drafts`, `login_attempts`, `reschedule_requests`, `exam_reschedule_requests`, `notifications`) that are created on-demand via the idempotent `ensure_*_table()` / `ensure_*_columns()` functions in `core/automation.php` — so a real production database typically has ~30 tables.

The schema-declared tables are:

```

users          applicant_exam_slots  course_caps
departments    interview_slots        course_passing_scores
course_departments  interview_queue      custom_courses
department_schedules  reschedule_logs     school_settings
applicants      admission_results    audit_logs
documents       course_suggestions    password_resets
exams           sessions
exam_sections   notifications
questions       document_validations
exam_results
exam_slot_schedule

```

The on-demand auxiliary tables (created by `ensure_*` helpers in `core/automation.php`):

```

notifications      (re-declared idempotently)
login_attempts
exam_drafts
reschedule_requests
exam_reschedule_requests

```

`ensure_email_verification_columns()` also adds the email-verify columns (`email_verified`, `email_verify_token`, ...) on first boot.

A3. Is the schema normalized?

Yes — to **3NF** (third normal form), with deliberate denormalization in two places for performance:

- `applicants.overall_status` is a denormalized “current state” column that mirrors what could be derived by joining `documents` + `exam_results` + `interview_queue` + `admission_results`. It’s updated atomically in the same transactions as those other writes, so it never drifts.
- `interview_slots.booked` is a denormalized count of `interview_queue` rows in that slot. It’s incremented in the same `FOR UPDATE` transaction as the queue insert, so it never drifts.

These denormalizations exist to keep the hot-path dashboard queries single-table and fast.

A4. What about foreign keys?

~**28 FOREIGN KEY constraints** across the schema, with sensible `ON DELETE` behaviour:

- **ON DELETE CASCADE** for owned data — if an `applicant` row is deleted, their `documents`, `exam_results`, `interview_queue`, `admission_results` and `course_suggestions` rows go with them.

- **ON DELETE SET NULL** for actor references — if a staff user is deleted, their `reviewed_by` / `released_by` references become NULL (we keep the historical record but unbind the deleted actor).
- **ON DELETE RESTRICT** for reference data — courses, departments, school years cannot be deleted while applicants reference them; the admin must reassign first.

A5. What about unique constraints?

~13 **UNIQUE indexes** form the integrity backbone. The critical ones:

Index	What it prevents
<code>users.email</code>	Duplicate accounts.
<code>applicants.user_id</code>	Two applicant rows for the same user.
<code>documents (applicant_id, document_type)</code>	Duplicate document slots per applicant.
<code>applicant_exam_slots.applicant_id</code>	One exam slot per applicant.
<code>interview_queue.applicant_id</code>	One active interview row per applicant.
<code>admission_results.applicant_id</code>	One result per applicant.
<code>course_passing_scores (course, school_year)</code>	One threshold row per course-year.
<code>course_caps (course, school_year)</code>	One cap row per course-year.

These constraints turn “logic bugs” into “constraint violations” — duplicate inserts raise SQL state `23000` instead of silently producing inconsistent data, and the application catches them gracefully.

B. Triggers & Stored Procedures

B1. Do you use triggers or stored procedures?

No — by deliberate architectural choice, not by oversight.

All business logic lives in PHP (`core/automation.php` , `core/interview_scheduler.php` , `modules/<area>/*_action.php`). The database enforces integrity (foreign keys, uniqueness, transactions, NOT NULL constraints); the application enforces business rules.

Why this matters:

- **Portability** — the schema can move between MySQL / MariaDB / Aurora / Percona without rewriting database-resident logic.
- **Testability** — PHP unit tests can mock the data layer and assert business behaviour without a real database.
- **Observability** — every state change is visible in the application log, not buried inside an opaque trigger.
- **Reviewability** — business logic is in version control alongside the rest of the code, reviewable through normal PR workflow.

If a particular hot path ever becomes performance-bound, we can selectively introduce a stored procedure or trigger. The schema is ready for it (`mysqldump` already exports them).

C. Transactions, Locking & Deadlock Prevention

C1. How are concurrent writes handled?

Every multi-step state change is wrapped in a single transaction:

```
$pdo->beginTransaction();
try {
    // ... prepared statement 1
    // ... prepared statement 2
    $pdo->commit();
} catch (Throwable $e) {
    $pdo->rollBack();
    throw $e;
}
```

InnoDB defaults to **REPEATABLE READ** isolation, which is suitable for our workload. On the hot paths where we need a stricter guarantee (interview slot assignment, exam slot assignment), we explicitly use `SELECT ... FOR UPDATE` to acquire row-level write locks.

C2. How does interview-slot assignment avoid double-booking?

`core/interview_scheduler.php::assign_interview_slot()` looks like:

```

BEGIN;
SELECT id FROM interview_queue
WHERE applicant_id = ?
AND status NOT IN ('completed', 'no_show', 'cancelled')
FOR UPDATE;

SELECT s.id, s.capacity, s.booked, s.slot_date, s.slot_time
FROM interview_slots s
WHERE s.department = ?
AND s.is_active = 1
AND s.booked < s.capacity
AND s.slot_date >= CURDATE()
ORDER BY s.booked ASC, s.slot_date ASC, s.slot_time ASC, s.id ASC
FOR UPDATE;

-- Pick first row, then:
INSERT INTO interview_queue
(applicant_id, slot_id, queue_number, status, checked_in_at)
VALUES (?, ?, ?, 'checked_in', NOW());

UPDATE interview_slots
SET booked = booked + 1
WHERE id = ?;

UPDATE applicants
SET overall_status = 'interview'
WHERE id = ?;
COMMIT;

```

Three layers of protection:

1. **SELECT ... FOR UPDATE** acquires a row-level write lock on both the candidate's existing queue row (if any) and the open slots. Concurrent assigners block here until the first one commits.
2. **Predictable lock ordering** (`booked ASC, slot_date ASC, slot_time ASC, id ASC`) — every transaction acquires locks in the same order, so cyclic-wait deadlocks are impossible.
3. **UNIQUE on interview_queue.applicant_id** acts as a safety net: even if the above somehow allowed two concurrent inserts, only one would succeed and the other would raise SQL state `23000`.

C3. What if two staff approve the same document at once?

The `UPDATE documents SET status = 'approved'` is idempotent — running it twice has the same effect. The downstream auto-advance carries a guard:

```
UPDATE applicants
  SET overall_status = 'exam', ...
 WHERE id = ?
  AND overall_status NOT IN ('exam','interview','released')
```

so the second approver doesn't re-trigger the auto-assignment. The second approver sees their action succeed silently, but no duplicate exam slot is assigned and no duplicate notification is sent.

C4. How is the bulk Cancel & Move flow protected?

`modules/exam/staff_cancel_slot.php` and `modules/interview/staff_cancel_slot.php` both follow the same shape:

```
BEGIN;
SELECT id, capacity, booked FROM exam_slot_schedule
  WHERE id IN (:source_id, :replacement_id)
  FOR UPDATE;
-- Re-validate replacement capacity inside the transaction.
UPDATE applicant_exam_slots
  SET slot_id = :replacement_id, updated_at = NOW()
  WHERE slot_id = :source_id;
UPDATE exam_slot_schedule SET booked = ... WHERE id IN (...);
-- Audit log + per-applicant notification rows.
COMMIT;
```

Locking both the source and replacement slot rows up front prevents:

- Concurrent moves into the same replacement from oversubscribing it.
- Concurrent students booking into the source after the move started.

C5. What's the locking strategy for result-release?

`modules/results/staff_action.php` is the single-applicant code path:

1. `BEGIN`.
2. `INSERT INTO admission_results (applicant_id, result, remarks, released_by, released_at)` — UNIQUE on `applicant_id` here means a second concurrent insert will raise `23000` and roll back.
3. `UPDATE applicants SET overall_status = 'released' WHERE id = ?`.
4. `COMMIT`.

Bulk release (`staff_bulk.php`) does the same in a loop, one transaction per applicant, so a single failure doesn't roll back the entire batch.

D. Indexing & Performance

D1. What's the secondary-index strategy?

Every column that appears in a `WHERE` / `ORDER BY` / `JOIN` predicate has an index. Roughly:

- `applicants` — indexes on `user_id`, `course_applied`, `overall_status`, `school_year`, `(school_year, overall_status)`, `(course_applied, overall_status)`.
- `documents` — `(applicant_id, status)`, `(applicant_id, document_type)`.
- `exam_results` — `applicant_id`, `(applicant_id, exam_id)`.
- `interview_queue` — `(slot_id, status)`, `applicant_id`.
- `audit_logs` — `actor_id`, `action`, `entity`, `created_at`.

D2. How are the dashboard funnel queries optimized?

The funnel chart and the per-course stats use a single `GROUP BY overall_status` query against `applicants`, hitting the `(school_year, overall_status)` index. With a few thousand applicants this runs in single-digit milliseconds.

D3. What about the document-review queue?

The queue selects `applicants WHERE overall_status IN ('documents', 'submitted')`, joined to a count of pending documents. Both the status filter and the documents subquery hit indexed columns.

D4. What's the slowest query and why?

The admin export from `/admin/results` joins `applicants` ↔ `users` ↔ `admission_results` ↔ `exam_results` ↔ `documents`. It's not on a hot path (staff click it occasionally), and we use it as a streamed CSV write so memory stays flat.

E. Data Integrity & Consistency

E1. How do you prevent duplicate applications?

Application-layer check at registration: the same `first_name + last_name + birthdate` triplet is rejected. We don't add a `UNIQUE` constraint on this because rare legitimate collisions (twins with the same first name) would otherwise be permanently blocked — but the check catches the common “user clicked Submit twice” case.

A `UNIQUE` on `users.email` and on `applicants.user_id` catches the infrastructure case: two applicant rows for one human are impossible at the DB level.

E2. How does cascade delete behave?

- Deleting an `applicant` cascades to `documents`, `exam_results`, `interview_queue`, `admission_results`, `course_suggestions`, `applicant_exam_slots`, `notifications`.
- Deleting an `exam` cascades to `exam_sections`, `questions`, `exam_results`. We almost never delete exams in practice — we just toggle `is_active`.
- Deleting an `interview_slot` is `ON DELETE RESTRICT` if it has queue rows — staff must Cancel & Move first, which moves the rows out of the slot.

E3. How is the `overall_status` state machine enforced?

The application enforces forward-only transitions:

```
pending → documents → submitted → exam → interview → released
                                           ↘
                                           withdrawn (terminal)
```

Every transition is gated by a `WHERE overall_status ...` clause that rejects out-of-order updates. For example, auto-advance after document approval carries `AND overall_status NOT IN ('exam', 'interview', 'released')` — so accidentally re-approving the same document on an applicant who’s already at the exam stage doesn’t bounce them back to “all docs approved → assign exam slot”.

F. Backup, Recovery & Operations

F1. How do you back up the database?

`mysqldump` on a schedule, run from cron / systemd timer:

```
mysqldump -u root -p --single-transaction --routines --triggers \
  --databases plp_admissions > backup_$(date +%F).sql
```

- `--single-transaction` takes a consistent snapshot without locking tables — important because InnoDB supports it.
- `--routines --triggers` are included for completeness even though we don’t currently use either (so backups remain forward-compatible).

For larger deployments, logical dumps (`mysqldump`) are paired with binary backups (`xtrabackup` or filesystem snapshots) for faster restore.

F2. How do you restore?

```
mysql -u root -p plp_admissions < backup_2025-11-01.sql
```

Caveat: `database/schema.sql` is **destructive** — it `DROP`s every table before recreating them. Never run `schema.sql` against a production database without a backup first.

F3. How do you handle migrations?

Migrations today are managed by **forward-only patches inside** `core/automation.php` — small `ALTER TABLE ... ADD COLUMN IF NOT EXISTS ...` calls behind `ensure_*_columns()` and `ensure_*_table()` functions. Examples:

- `ensure_email_verification_columns()` — adds the email-verify columns idempotently.
- `ensure_notifications_table()` — creates `notifications` if missing.
- `ensure_login_attempts_table()` — creates `login_attempts` if missing.
- `ensure_exam_drafts_table()` — creates `exam_drafts` if missing.
- `ensure_reschedule_requests_table()` — creates `reschedule_requests` if missing.
- `ensure_exam_reschedule_requests_table()` — creates `exam_reschedule_requests` if missing.

This is a pragmatic alternative to a full migration framework like Phinx or Doctrine Migrations. For a larger team we'd move to a numbered-migration directory.

G. Security

G1. How do you prevent SQL injection?

100% prepared statements with parameter binding. Every query in the codebase uses `$pdo->prepare(...)->execute([$param1, $param2])`. We never concatenate user input into SQL.

To prove it: a code-level audit shows zero instances of string-interpolated query construction outside of fully internal `ORDER BY column ASC` style ordering, where the column name is whitelisted before substitution.

G2. How are passwords stored?

`password_hash($plain, PASSWORD_BCRYPT, ['cost' => 12])`. Stored in `users.password_hash`. Verified on login with `password_verify()`. No plaintext is ever logged.

Bcrypt at cost 12 is currently ~250 ms per hash on commodity hardware — fast enough for the user but slow enough to make brute force impractical.

G3. How do you protect the database connection?

- Credentials live in `.env` (loaded at boot, not committed to git).
- `config/db.php` builds a PDO DSN at runtime — no hardcoded passwords anywhere in source.
- Non-localhost connections are forced to use SSL (`PDO::MYSQL_ATTR_SSL_CA`).
- Database user permissions should be least-privilege: the app user has `SELECT, INSERT, UPDATE, DELETE` on the `plp_admissions` schema and nothing else — no `DROP`, no `GRANT`. The `schema.sql` import is run by a privileged DBA user, separately.

G4. Is the audit log tamper-proof?

The audit log is **append-only by application convention** — the app never `UPDATE`s or `DELETE`s rows in `audit_logs`. However, a DBA with raw SQL access can edit it. For high-stakes deployments we'd recommend either:

- Mirroring `audit_logs` to an external SIEM / log store as it's written.
- Using a database-level `AFTER UPDATE` trigger on `audit_logs` that prevents updates (yes — this would be a good use of a trigger).
- Cryptographically chaining log rows (each row's hash includes the previous row's hash).

Currently none of those is implemented; for the school's threat model, application-layer append-only is sufficient.

G5. What sensitive data is in the database?

Sensitivity	Data	Storage
High	passwords	bcrypt-hashed in <code>users.password_hash</code>
High	password reset tokens	random 32-byte tokens, expiring in 1h, in <code>password_resets</code>
High	email verification codes	random 6-digit codes + 32-byte magic-link tokens, expiring in 1h
Medium	personally identifiable info (name, birthdate, address, phone)	plaintext in <code>users</code> / <code>applicants</code>
Medium	student exam answers	plaintext in <code>exam_results.answers</code> (JSON)
Low	audit logs	plaintext

For GDPR / Data Privacy Act compliance, sensitive PII should be encrypted at rest (filesystem-level encryption is the minimum). Today we rely on OS-level disk encryption for that layer.

H. Concurrency & Live Operations

H1. What happens if two staff approve the same document at once?

See **C3** above. Idempotent UPDATE + guarded auto-advance means the second approver is a no-op.

H2. What happens if a student submits the exam twice?

The exam submission is wrapped in a transaction and finishes with an `INSERT INTO exam_results`. There's no `UNIQUE` constraint on `(applicant_id, exam_id)` today — relying instead on the application setting `overall_status = 'interview'` and the next GET to `/student/exam` redirecting to the result page.

Honest gap, worth flagging: add `UNIQUE KEY uq_applicant_exam (applicant_id, exam_id)` to `exam_results` for defence-in-depth.

H3. How do you handle concurrent results release?

See **C5** above. `UNIQUE` on `admission_results.applicant_id` plus per applicant transactions means a second concurrent release raises `23000` and rolls back.

H4. What about the bulk Cancel & Move flow?

See **C4** above. `FOR UPDATE` on source + replacement slots prevents oversubscription under concurrent moves.

H5. What about session storage?

`core/Session.php` uses PHP's native session handler, backed by the OS filesystem. The `sessions` table exists in the schema but is currently unused — it's there for a future move to DB-backed sessions when we deploy to multiple PHP-FPM nodes behind a load balancer.

H6. How many concurrent users can the DB handle?

Realistically several hundred concurrent staff + several thousand applicants doing reads. The hot path is the exam-submit endpoint — a single POST writes ~5 rows and runs one transaction, which InnoDB handles at thousands of TPS on commodity hardware.

The bottleneck under heavy load would likely be PHP-FPM workers, not the database.

I. Reporting & Analytics

I1. Can you export data?

Yes — `/admin/dashboard` and `/admin/results` both support CSV export with filters (date range, status, course, school year). The export builds a streamed CSV using the same `SELECT` that powers the dashboard list.

I2. Can you join with the SIS / registrar after admission?

That's a planned integration. The natural join key is `applicants.id` combined with the `school_year`. Today the export is CSV-based; a real-time API endpoint would be a future addition.

J. Role-Specific Q&A

J1. Why is SSO blocked from `/staff/results`?

In the role redesign, the team explicitly separated **operations** (SSO) from **decision-making** (Dean / Admin). The release of an admission result is a decision with consequences (cap consumption, enrollment intent commitment), so the role that schedules and reviews documents should not be the same role that signs off on results.

The guard is the auth header on the page itself:

```
Auth::requireRole(ROLE_DEAN, ROLE_ADMIN);
```

If SSO navigates to `/staff/results` they get redirected with a permission flash. The sidebar for SSO also hides the link.

J2. Why does the Dean's override require a written reason?

So that decisions which contradict a Professor's recommendation are auditable. The flow:

1. Dean clicks **Accept** on a Recommended: Decline row (or vice versa).
2. The front-end pops the override-reason modal.
3. Submit requires a non-empty reason.

4. The handler stores the reason on `admission_results.remarks` and writes an `admission_result_override` row to `audit_logs`.

This is the difference between “the system made the decision for me” and “I overrode the recommendation, and here’s why”.

J3. Why can the Proctor only see their own college’s exam rooms?

Because the role exists for **exam-day operations only**. The Proctor is the person physically in the room reading the access code aloud. They have no need to see other colleges’ rooms, and the scope rule prevents a sloppy click from generating a code for the wrong room.

The check is in `staff_slots.php`:

```
if ($isProctor && $slotDept !== '' && $slotDept === $staffDept) {  
    return true;  
}  
return false;
```

K. Tough / Adversarial Questions

K1. “Your system doesn’t use triggers or stored procedures. Isn’t that bad practice?”

Calmly: “It’s a deliberate architectural choice, not an oversight. Modern application frameworks consistently move business logic out of the database and into the application layer for portability, testability, and observability. The database is where we enforce integrity (foreign keys, uniqueness, transactions, NOT NULL constraints) — and we use ~28 foreign keys and ~13 unique constraints to do that — but business logic like ‘when an exam is passed, schedule an interview’ lives in version-controlled, unit-testable PHP code where developers can read it. If we ever need a SQL-level audit trigger or a heavy stored procedure for performance, the schema is ready for it.”

K2. “Could the database be hacked?”

Possible attack surfaces, with mitigations:

Attack	Mitigation
SQL injection	100% prepared statements
Direct connection	DB bound to localhost or VPN; only the app user reaches it
Stolen <code>.env</code>	<code>.env</code> is gitignored, mode 600, separate from source
Lost backup	Backups should live on an encrypted external store (operational, not code)
Password brute force	bcrypt cost 12 + login rate limiting (5 attempts → 15-min lockout)
CSRF on POST	<code>csrf_field()</code> on every form, token verified server-side

K3. “What if your `users` table grows to 100,000 rows?”

Login is `WHERE email = ?`; with `UNIQUE KEY uq_email`, that’s an $O(\log n)$ lookup. 100k rows is roughly 17 index reads — still sub-millisecond on InnoDB. We’re far from any scale limit.

K4. “Show me a query that shows the funnel.”

```
SELECT overall_status, COUNT(*) AS n
FROM applicants
WHERE school_year = '2026-2027'
GROUP BY overall_status
ORDER BY FIELD(overall_status,
               'pending','documents','submitted',
               'exam','interview','released','withdrawn');
```

This is exactly the query behind the admin dashboard’s funnel chart.

K5. “What if the server crashes during the entrance exam?”

The exam autosaves answers every 60 seconds via `POST /api/exam-autosave` into `exam_drafts`. On reload, drafts are restored client-side. The final submission writes to `exam_results` in a single transaction — so the worst case is a student loses the last ≤ 60 seconds of answers, not the entire exam.

K6. “How do I know the audit log isn’t lying?”

Each row stores: `actor_id` (FK to `users`), `action`, `description`, `entity` + `entity_id`, `ip_address`, `created_at`. It’s append-only by application convention. For higher assurance, we’d recommend shipping logs off-box to an external store; that’s an operational hardening step, not an app change.

L. Quick-Reference Talking Points (One-Liners)

For the rapid-fire Q&A round.

Question	One-line answer
Database engine?	MariaDB / MySQL with InnoDB and utf8mb4.
How many tables?	~26 schema-declared + ~5 on-demand (<code>exam_drafts</code> , <code>login_attempts</code> , <code>reschedule_requests</code> , <code>exam_reschedule_requests</code> , <code>notifications</code>).
Triggers?	None — business logic is in PHP for portability and testability.
Stored procedures?	None — equivalent logic is in <code>core/automation.php</code> wrapped in transactions.
Deadlock prevention?	Predictable lock order + short transactions + UNIQUE constraints as a safety net.
Isolation level?	InnoDB default — REPEATABLE READ with <code>SELECT ... FOR UPDATE</code> on hot paths.
SQL injection protection?	100% prepared statements via PDO.
Password storage?	bcrypt at cost 12.
Foreign keys?	~28 FKs with CASCADE for owned data, SET NULL for actor references.
Unique constraints?	~13, including the critical <code>interview_queue.applicant_id</code> and <code>admission_results.applicant_id</code> .
Audit logging?	Every state change writes to <code>audit_logs</code> with actor, IP, timestamp.
Backups?	<code>mysqldump --single-transaction</code> on schedule; restore via SQL import.
Migrations?	Forward-only idempotent <code>ALTER TABLE</code> / <code>CREATE TABLE IF NOT EXISTS</code> patches behind functions in <code>core/automation.php</code> .
Who releases results?	Dean (per college) and Admin (system-wide). SSO is locked out.
What does override require?	A non-empty written reason, stored on <code>admission_results.remarks</code> and audited.

Built for the PLP Admissions presentation. Verified against `baseline` after the latest project zip merge. If anyone challenges a claim during Q&A, the codebase is the source of truth.